

AI / ML Interview - Khái niệm & Câu hỏi

1. LLM là gì?

LLM là viết tắt của Large Language Model.

Đây là một loại mô hình AI được huấn luyện trên lượng dữ liệu văn bản khổng lồ để hiểu và tạo ra văn bản giống con người. Các mô hình này học các pattern trong ngôn ngữ và có thể thực hiện các tác vụ như trả lời câu hỏi, tóm tắt, dịch thuật, viết code, v.v.

LLM → Large Language Model

AI → Artificial Intelligence

Model → Một hệ thống được huấn luyện để học các pattern

Generate → Tạo ra nội dung mới

Exam Tip

Bắt đầu bằng một định nghĩa rõ ràng, thêm các ý chính, sử dụng ví dụ nếu có thể, và trình bày ngắn gọn.

Important Fact: GPT-4 là một LLM hàng đầu được phát triển bởi OpenAI.

2. Token là gì?

Token là đơn vị nhỏ nhất của văn bản mà một mô hình có thể hiểu được. Nó có thể là một từ, một subword, hoặc thậm chí một ký tự đơn lẻ.

Example: "unbelievable" → "un", "believ", "able"

3. Tokenization là gì?

Tokenization là quá trình chuyển đổi văn bản thành các token để mô hình có thể xử lý được.

Common Tokenizers: BPE, WordPiece, SentencePiece

4. Embedding là gì?

Embedding là một biểu diễn số của văn bản (hoặc dữ liệu khác) trong một không gian vector liên tục. Nó giúp mô hình hiểu được ý nghĩa ngữ nghĩa (semantic meaning).

Example: Các từ "king" và "queen" sẽ có vector gần nhau.

Tokenization to Embedding (3-Step Flow)



5. Transformer là gì?

Transformer là một kiến trúc mô hình deep learning được giới thiệu trong bài báo "Attention Is All You Need" của Vaswani et al., 2017. Nó hoàn toàn dựa trên attention mechanism, không sử dụng RNN hay CNN, và rất hiệu quả cho các tác vụ sequence-to-sequence như language modeling, translation, summarization, v.v.

Key Components: Multi-Head Attention, Feed-Forward Network, Add & Norm (Residual + LayerNorm), Positional Encoding.

Exam Tip

Luôn đề cập ý chính + một component hoặc ưu điểm quan trọng.

6. Vì sao Transformer gần như thay thế hoàn toàn RNN/LSTM cho LLM?

Transformer thay thế RNN/LSTM vì:

- Chúng sử dụng **self-attention**, giúp nắm bắt các dependency tầm xa tốt hơn.
- Chúng có thể **parallelizable**, giúp huấn luyện nhanh hơn nhiều trên các tập dữ liệu lớn.
- Chúng xử lý **long sequences** hiệu quả hơn.
- Khả năng **scalability** và hiệu năng tốt hơn ở quy mô lớn.

Important Fact

Transformer đã giúp huấn luyện được các mô hình lớn hơn nhiều (BERT, GPT, T5, PaLM, v.v.) - những mô hình đang vận hành các LLM hiện đại.

7. Self-attention là gì?

Self-attention là một cơ chế cho phép một token trong chuỗi chú ý (attend) đến tất cả các token khác trong chuỗi để tính toán biểu diễn của nó. Nó giúp mô hình hiểu được ngữ cảnh và mối quan hệ giữa tất cả các từ, bất kể khoảng cách giữa chúng.



Exam Tip

Sử dụng một ví dụ đơn giản hoặc diagram để giải thích self-attention.

8. Query, Key, và Value trong attention là gì?

Trong attention, mỗi token được chuyển đổi thành ba vector:

Query (Q). → Token đang tìm kiếm điều gì.

Key (K). → Token chứa thông tin gì (dùng để so khớp).

Value (V). → Thông tin thực tế cần được tổng hợp.

How it works:



9. Multi-head attention là gì?

Multi-head attention chạy nhiều self-attention operation song song với các phép chiếu (projection) đã học khác nhau. Điều này cho phép mô hình tập trung vào các phần khác nhau của input và nắm bắt các loại quan hệ khác nhau cùng lúc.

Why it helps: Nó cải thiện khả năng của mô hình trong việc hiểu các pattern và dependency phức tạp.

How it works:



Exam Tip

Hãy nghĩ mỗi head như đang nhìn chuỗi từ một góc nhìn khác nhau. Kết hợp lại, chúng mang đến một sự hiểu biết phong phú hơn.

10. Positional encoding là gì?

Positional encoding là cách thêm thông tin về vị trí của các token trong chuỗi, vì Transformer không có khái niệm về thứ tự vốn có như RNN. Nó được cộng vào input embeddings.

Common methods:

1. Sinusoidal positional encoding (cố định)
2. Learned positional encoding (có thể huấn luyện).

Important Fact

Ashish Vaswani et al. đã giới thiệu Transformer trong bài báo "Attention Is All You Need" (2017).

11. Sự khác biệt giữa Transformer encoder-only, decoder-only, và encoder-decoder là gì?

Encoder-Only	Decoder-Only	Encoder-Decoder
• Dùng cho các tác vụ hiểu (understanding) như classification, NER, search, embeddings. • Example: BERT	• Dùng cho các tác vụ sinh văn bản (text generation). • Sinh từng token một. • Example: GPT	• Dùng cho các tác vụ sequence-to-sequence như translation, summarization. • Example: T5
Input → Encoder → Output (Representation)	Input → Decoder → Output (Generated Text)	Input → Encoder → Decoder → Output (Generated Text)

Exam Tip

BERT đọc toàn bộ câu.
GPT viết từng từ một. T5 làm cả hai!

Key Difference: Attention được áp dụng ở đâu và cách mô hình sử dụng ngữ cảnh để tạo ra output.

12. Context window là gì?

Context window là số lượng token tối đa mà một LLM có thể xem xét trong một lần (bao gồm cả input và token được sinh ra). Nó xác định lượng văn bản mà mô hình có thể "nhìn thấy" tại một thời điểm.

Example: Một mô hình với context window 8K có thể xử lý tối đa 8.192 token trong một request.

Exam Tip

Context window càng lớn cho phép mô hình xử lý tài liệu dài hơn và duy trì nhiều lịch sử hơn trong hội thoại.

13. Điều gì xảy ra khi input vượt quá context window?

Nếu input dài hơn context window của mô hình:

- Mô hình sẽ cắt bớt (truncate) một phần của input.
- Thông tin quan trọng ở đầu có thể bị mất.
- Điều này có thể dẫn đến hiểu sai hoặc trả lời không chính xác.

Solution: Sử dụng các kỹ thuật như chunking, summarization, hoặc Retrieval-Augmented Generation (RAG) để xử lý input dài.

Important Fact

Hầu hết các mô hình không báo lỗi; chúng chỉ đơn giản bỏ qua các token vượt quá context window.

14. Model parameters là gì?

Model parameters là các trọng số (weights) và độ lệch (biases) có thể học được trong một mô hình, được cập nhật trong quá trình huấn luyện. Chúng lưu trữ kiến thức học được từ dữ liệu.

Example: Trong một Transformer, parameters tồn tại trong attention layers, feed-forward layers, embeddings, v.v.

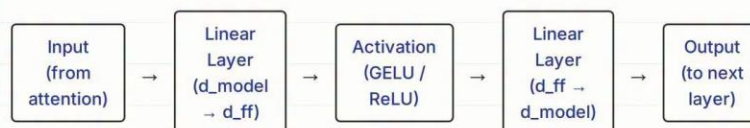
Important Fact

Càng nhiều parameters thường đồng nghĩa với khả năng (capacity) lớn hơn, nhưng cũng cần nhiều compute và memory hơn.

15. Feed-forward network bên trong Transformer là gì?

Feed-Forward Network (FFN) là một mạng fully connected được áp dụng cho từng vị trí một cách riêng biệt và giống hệt nhau. Nó bao gồm hai linear layer với một activation phi tuyến ở giữa (thường là GELU hoặc ReLU).

Structure:



Exam Tip

Hãy nghĩ FFN như "bước suy nghĩ của bộ não" sau khi chú ý (attention).

16. Residual connections là gì?

Residual connections (hay skip connections) cho phép input của một layer (hoặc một khối layer) được cộng vào output của nó. Chúng giúp huấn luyện các mô hình rất sâu bằng cách giảm vấn đề vanishing gradient và cải thiện luồng thông tin.

How it works: Input được "bỏ qua" (skip) một hoặc nhiều layer và cộng vào output.

Formula: $\text{Output} = F(x) + x$

trong đó $F(x)$ là output của các layer, và x là input.



Exam Tip

Residual connections giúp việc huấn luyện các mạng sâu hơn dễ dàng hơn và thường mang lại hiệu năng tốt hơn.

17. Normalization, ví dụ LayerNorm/RMSNorm, là gì?

Các kỹ thuật normalization giúp ổn định và tăng tốc quá trình huấn luyện bằng cách chuẩn hóa các activation.

- **LayerNorm:** Chuẩn hóa qua các feature trong cùng một sample (tức là qua hidden dimension). Phổ biến trong Transformer.
- **RMSNorm:** Một biến thể của LayerNorm, chuẩn hóa theo root mean square của activation, không trừ đi mean. Tính toán đơn giản hơn một chút và thường được dùng trong LLM.

Why used?

Chúng giảm internal covariate shift và giúp huấn luyện nhanh hơn, ổn định hơn, ít nhạy cảm hơn với việc khởi tạo (initialization).

Important Fact

LayerNorm sử dụng cả mean và variance.
RMSNorm chỉ sử dụng RMS (không trừ mean).

18. Hàm softmax là gì?

Softmax chuyển đổi một vector số thực (logits) thành một phân phối xác suất, trong đó tất cả xác suất nằm trong khoảng 0 đến 1 và có tổng bằng 1.

Formula:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

Use: Trong LLM, softmax được dùng trên toàn bộ vocabulary để chuyển logits thành xác suất cho token tiếp theo.

Exam Tip

Softmax khuếch đại các giá trị lớn và làm giảm các giá trị nhỏ, biến điểm số (logits) thành xác suất.

19. Logits là gì?

Logits là các điểm số thô (raw), chưa chuẩn hóa được mô hình xuất ra trước khi áp dụng softmax. Chúng có thể là bất kỳ số thực nào (dương hoặc âm). Softmax chuyển các logits này thành xác suất trong khoảng 0 và 1.

Example: Với vocabulary gồm 4 token, mô hình có thể xuất ra logits [2.1, 0.5, -1.3, 3.0]. Sau softmax → xác suất như [0.24, 0.11, 0.03, 0.62].



Exam Tip

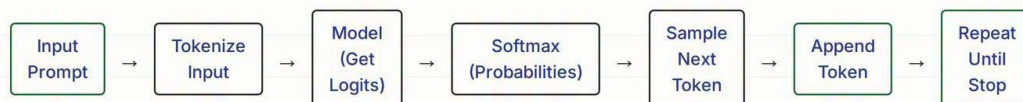
Hãy nhớ: logits là điểm tự tin (confidence scores) của mô hình trước khi chuẩn hóa. Softmax biến chúng thành xác suất.

20. LLM sinh ra token tiếp theo như thế nào?

LLM sinh văn bản từng token một theo cách autoregressive.

Steps:

1. Lấy input prompt và tokenize nó.
2. Mô hình xử lý các token và tạo ra logits cho token tiếp theo.
3. Áp dụng softmax để có xác suất trên toàn bộ vocabulary.
4. Chọn token tiếp theo (ví dụ, dùng greedy, top-k, hoặc top-p sampling).
5. Nối (append) token đã chọn vào chuỗi.
6. Lặp lại bước 2-5 cho đến khi đạt điều kiện dừng (ví dụ, token <eos> hoặc đạt độ dài tối đa).



Exam Tip

LLM có tính autoregressive: chúng dự đoán token tiếp theo chỉ dựa trên các token trước đó. Từng bước một!

21. Pretraining là gì?

Pretraining là giai đoạn ban đầu, nơi mô hình được huấn luyện trên một lượng lớn dữ liệu văn bản chưa gán nhãn để học các pattern và biểu diễn ngôn ngữ tổng quát.

Mục tiêu thường là dự đoán token tiếp theo (causal language modeling) hoặc điền vào các token bị thiếu (masked language modeling).

Example: Dòng GPT sử dụng causal language modeling.

BERT sử dụng masked language modeling.

Goal: Học một sự hiểu biết rộng về ngôn ngữ, sự kiện, cú pháp (syntax), ngữ nghĩa (semantics), và kiến thức thế giới.

Important Fact

OpenAI GPT-3 được pretrain trên khoảng 45 TB dữ liệu văn bản từ sách, website, và các nguồn khác.

Typical Pretraining Pipeline:



22. Overfitting là gì và làm sao để ngăn chặn nó?

Overfitting xảy ra khi một mô hình học dữ liệu huấn luyện quá kỹ, bao gồm cả nhiễu của nó, và hoạt động kém trên dữ liệu chưa từng thấy.

How to prevent:

- Lấy thêm training data.
- Sử dụng regularization (L1, L2).
- Áp dụng dropout.
- Sử dụng early stopping.
- Giữ mô hình đơn giản.
- Sử dụng cross-validation.



23. Sự khác biệt giữa bias và variance là gì?

- **Bias:** Lỗi do các giả định quá đơn giản trong mô hình.
Bias cao → underfitting.
- **Variance:** Lỗi do sự nhạy cảm với các biến động nhỏ trong training data.
Variance cao → overfitting.

Analogy: Bias đến từ các giả định sai, variance đến từ việc phản ứng thái quá với dữ liệu.

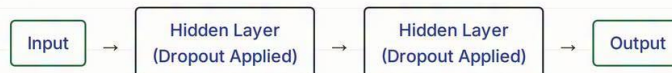
Important Fact

Các mô hình tốt nhất đạt được sự cân bằng hợp lý: bias thấp và variance thấp. Đây chính là mục tiêu!

24. Dropout là gì?

Dropout là một kỹ thuật regularization, trong đó, trong quá trình huấn luyện, các neuron ngẫu nhiên bị "loại bỏ" (đặt về 0) với xác suất p . Điều này ngăn chặn sự co-adaptation giữa các neuron và giảm overfitting.

How it works:



Chỉ được dùng trong quá trình huấn luyện. Khi inference, tất cả neuron được sử dụng với trọng số đã được điều chỉnh (scaled).

Exam Tip

Tỷ lệ dropout p thường trong khoảng 0.1 đến 0.5. Đừng quên tắt dropout khi inference!

25. Learning rate là gì?

Learning rate (α) kiểm soát mức độ mà trọng số của mô hình được cập nhật trong quá trình huấn luyện.

Update rule: $w_{\text{new}} = w_{\text{old}} - \alpha \times \text{gradient}$

Choosing the right learning rate:

Too Low	Just Right	Too High
Huấn luyện rất chậm, có thể bị mắc kẹt.	Hội tụ mượt mà và hiệu quả.	Huấn luyện không ổn định, có thể phân kỳ.

Important Fact

Hầu hết optimizer (SGD, Adam, v.v.) sử dụng learning rate để kiểm soát việc cập nhật trọng số. Đây là một trong những hyperparameter quan trọng nhất!

26. Label smoothing là gì và vì sao nó được sử dụng?

Label smoothing ngăn mô hình trở nên quá tự tin (overconfident) bằng cách làm mềm các target labels. Thay vì dùng 1 cho lớp đúng và 0 cho các lớp khác, ta dùng một giá trị nhỏ ϵ cho tất cả các lớp.

Formula: $y_{\text{smooth}}(i) = 1 - \epsilon$ nếu i là lớp đúng, ngược lại $\epsilon / (K - 1)$

trong đó K = số lượng classes và ϵ là một giá trị nhỏ (ví dụ, 0.1).

Why used?:

- Giảm sự overconfidence.
- Cải thiện generalization.
- Hữu ích khi labels bị nhiễu (noisy).
- Thường tăng độ mạnh mẽ (robustness) của mô hình.

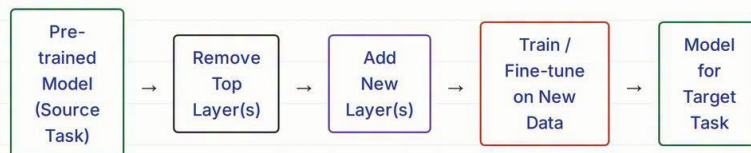
Before (Hard Labels)				After (Smoothed Labels)			
A	B	C	D	A	B	C	D
1.0	0	0	0	0.9	0.033	0.033	0.033

27. Transfer learning là gì?

Transfer learning là một kỹ thuật, trong đó một mô hình đã học trên một tác vụ (source task) được tái sử dụng làm điểm khởi đầu cho một tác vụ khác (target task).

How it works:

- Sử dụng một pre-trained model (ví dụ, đã huấn luyện trên ImageNet).
- Loại bỏ (các) layer cuối cùng của mô hình đó.
- Thêm (các) layer mới cho tác vụ cụ thể của bạn.
- Fine-tune mô hình trên tập dữ liệu của riêng bạn.



Benefits: Tiết kiệm thời gian, cần ít dữ liệu hơn, hiệu năng tốt hơn, và giảm compute.

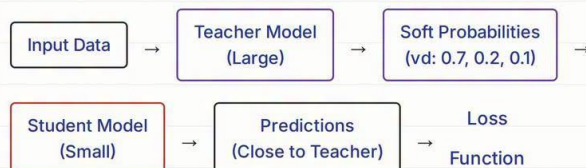
Important Fact

Transfer learning được sử dụng rộng rãi trong NLP (ví dụ, BERT), CV (ví dụ, ResNet), và các mô hình speech. Nó tận dụng kiến thức học được từ các tập dữ liệu lớn.

28. Knowledge distillation là gì?

Knowledge distillation là một kỹ thuật trong đó một student model nhỏ học cách bắt chước một teacher model lớn hơn. Student cố gắng khớp với các soft probabilities (output) của teacher thay vì các hard labels.

How it works:



Benefits: Student model trở nên nhỏ hơn và nhanh hơn.

Exam Tip

Distillation giống như việc một giáo viên truyền lại kiến thức cho học sinh dưới một hình thức đơn giản hơn.

29. Batch normalization là gì?

Batch Normalization là một kỹ thuật được dùng để chuẩn hóa input của mỗi layer trong một mini-batch. Nó giúp ổn định và tăng tốc quá trình huấn luyện.

How it works: Nó chuẩn hóa các activation để có mean bằng 0 và variance bằng 1, sau đó scale và shift chúng bằng các tham số có thể học được (learnable parameters).

$$\hat{x} = (x - \mu_B) / \sqrt{(\sigma_B^2 + \epsilon)} \quad , \quad y = \gamma \hat{x} + \beta$$

where: • x = input activation

• μ_B = batch mean

• σ_B^2 = batch variance

• γ, β = learnable parameters

• ϵ = hằng số nhỏ (vd, $1e-5$)

Benefits:

- Giảm internal covariate shift.
- Cho phép learning rate cao hơn.
- Đóng vai trò như một regularizer.
- Giúp huấn luyện hội tụ nhanh hơn.

Exam Tip

Batch Normalization được áp dụng sau linear layer và trước activation function (vd, ReLU). Nó được sử dụng rộng rãi trong các mạng sâu như CNN.

30. Gradient clipping là gì?

Gradient clipping là một kỹ thuật được dùng để ngăn chặn vấn đề exploding gradient trong RNN và các mạng sâu khác.

Why used? Khi gradient trở nên quá lớn, chúng có thể gây ra các update không ổn định và khiến mô hình phân kỳ (diverge).

How it works: Nếu norm của gradient vượt quá một ngưỡng, nó sẽ được scale xuống.

Nếu $\|g\| > c$, thì $g_{clipped} = g \times c / \|g\|$
trong đó g = gradient, c = ngưỡng clipping

Example: Nếu gradient norm = 25 và ngưỡng $c = 5$, thì gradient được scale xuống theo $5/25 = 0.2$.

Benefits:

- Ngăn chặn exploding gradients.
- Ổn định quá trình huấn luyện.
- Thường được dùng trong RNN và LSTM.

Exam Tip

Gradient clipping không thay đổi hướng của gradient, chỉ thay đổi độ lớn của nó. Nó giúp ích trong việc huấn luyện các mô hình recurrent sâu.

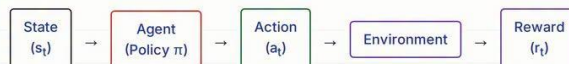
31. Reinforcement learning là gì?

Reinforcement Learning (RL) là một loại machine learning, trong đó một agent học cách đưa ra quyết định bằng cách tương tác với một environment để tối đa hóa cumulative reward.

Key components:

- **Agent:** Học và thực hiện các action.
- **Environment:** Thế giới mà agent tương tác.
- **State:** Tình huống hiện tại của environment.
- **Action:** Các lựa chọn có sẵn cho agent.
- **Reward:** Phản hồi nhận được sau khi thực hiện một action.

How it works:



Exam Tip

RL là tất cả về thử và sai! Agent học chính sách (policy) tốt nhất để tối đa hóa tổng reward theo thời gian.

32. Confusion matrix là gì?

Confusion matrix là một bảng được dùng để đánh giá hiệu năng của một mô hình classification. Nó so sánh các nhãn được dự đoán (predicted labels) với các nhãn thực tế (actual labels).

For Binary Classification:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Key Metrics:

- Accuracy = $(TP+TN)/(TP+TN+FP+FN)$
- Precision = $TP/(TP+FP)$
- Recall (Sensitivity) = $TP/(TP+FN)$
- F1 Score = $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

Why used?

- Cung cấp cái nhìn chi tiết về hiệu năng của mô hình.
- Hữu ích khi các lớp bị mất cân bằng (imbalanced).
- Giúp chọn đúng metric dựa trên loại bài toán.

Exam Tip

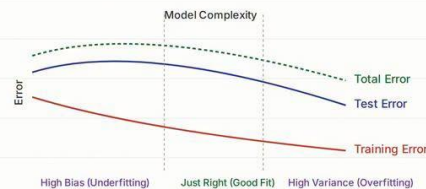
Confusion matrix giúp bạn hiểu mô hình của mình đang sai ở đâu. Luôn nhớ rõ TP, TN, FP, FN!

33. Bias-variance tradeoff là gì?

Bias-variance tradeoff là sự cân bằng giữa hai loại lỗi khiến mô hình không thể generalize tốt.

- **High Bias:** Mô hình quá đơn giản. Bị underfitting dữ liệu.
Dẫn đến training error cao và test error cao.
- **High Variance:** Mô hình quá phức tạp. Bị overfitting dữ liệu.
Dẫn đến training error thấp nhưng test error cao.

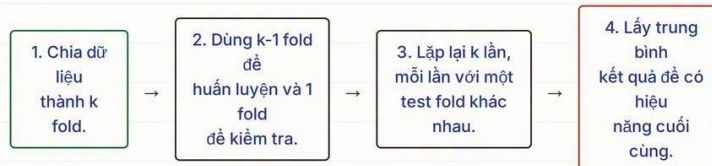
Goal: Tìm sự cân bằng hợp lý để tối thiểu hóa tổng lỗi (total error).



34. Cross-validation là gì?

Cross-validation là một kỹ thuật được dùng để đánh giá một mô hình bằng cách chia dữ liệu thành nhiều fold và huấn luyện/kiểm tra mô hình nhiều lần.

How it works? (k-Fold Cross-Validation)



Exam Tip

Cross-validation cho một cái nhìn tốt hơn về việc mô hình sẽ hoạt động ra sao trên dữ liệu chưa thấy so với một lần chia train/test đơn.

Benefits:

- Cung cấp một ước lượng đáng tin cậy hơn về hiệu năng mô hình.
- Sử dụng dữ liệu hiệu quả cho cả huấn luyện và kiểm tra.

35. Sự khác biệt giữa classification và regression là gì?

Cả hai đều là các tác vụ supervised learning nhưng khác nhau ở loại output mà chúng dự đoán.

	Classification	Regression
Output Type	Nhân/danh mục rời rạc	Giá trị liên tục
Example	Phát hiện spam (Spam/Not Spam)	Dự đoán giá nhà (\$)
Algorithms	Logistic Regression, SVM, Decision Trees, Naive Bayes	Linear Regression, Ridge, Lasso, Decision Trees
Evaluation Metrics	Accuracy, Precision, Recall, F1 Score, ROC-AUC	MAE, MSE, RMSE, R^2
Output Range	Số lượng class hữu hạn	Bất kỳ số thực nào

Exam Tip

Nếu biến mục tiêu là một category → đó là classification. Nếu là một con số (hoặc số lượng) → đó là regression.

Key Idea: Classification trả lời "Class nào?" trong khi regression trả lời "Bao nhiêu?"

36. Decision tree là gì?

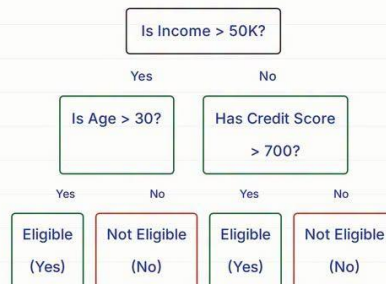
Decision tree là một cấu trúc giống flowchart, trong đó các internal node đại diện cho feature, các nhánh (branch) đại diện cho quy tắc quyết định, và leaf node đại diện cho output.

How it works:

- Bắt đầu tại root node (toàn bộ dữ liệu).
- Chia dữ liệu dựa trên feature tốt nhất.
- Lặp lại việc chia cho từng nhánh.
- Dừng lại khi đạt điều kiện dừng (vd, max depth, min samples).
- Các node cuối (leaves) đưa ra dự đoán.

Exam Tip

Decision tree dễ hiểu nhưng có thể bị overfit. Pruning và giới hạn depth giúp cải thiện generalization.



Key Points:

- Dễ diễn giải và trực quan hóa.
- Có thể xử lý cả dữ liệu số và dữ liệu phân loại (categorical).
- Dễ bị overfitting nếu không được pruning đúng cách.

37. Ensemble learning là gì?

Ensemble learning kết hợp nhiều mô hình để đạt hiệu năng tốt hơn bất kỳ mô hình đơn lẻ nào.

Types of Ensemble Methods:

Bagging (Bootstrap Aggregating)

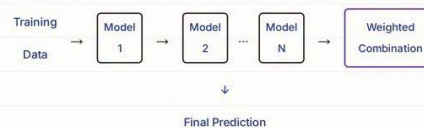
Huấn luyện nhiều mô hình trên các tập con dữ liệu ngẫu nhiên khác nhau song song và lấy trung bình các dự đoán của chúng.



Example: Random Forest

Boosting

Huấn luyện các mô hình tuần tự, mỗi mô hình tập trung vào lỗi của mô hình trước đó.



Example: AdaBoost, Gradient Boosting, XGBoost

38. Regularization là gì và vì sao nó cần thiết?

Regularization là một kỹ thuật được dùng để ngăn chặn overfitting bằng cách thêm một penalty term vào loss function dựa trên độ phức tạp của mô hình.

Common Types:

- L1 Regularization (Lasso):
Cộng thêm giá trị tuyệt đối của trọng số.
- L2 Regularization (Ridge):
Cộng thêm bình phương của trọng số.

L1 Regularization (Lasso):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n |\theta_i|$$

L2 Regularization (Ridge):

$$J(\theta) = \text{Loss}(\theta) + \lambda \sum_{i=1}^n \theta_i^2$$

where:

- λ = cường độ regularization (hyperparameter)
- θ_i = trọng số mô hình
- n = số lượng trọng số

Why needed?

- Giúp ngăn chặn overfitting.
- Cải thiện generalization đến dữ liệu chưa thấy.
- Khuyến khích mô hình đơn giản hơn và giảm variance.

Exam Tip

λ nhỏ $\rightarrow$ ít regularization hơn (mô hình phức tạp hơn). λ lớn $\rightarrow$ nhiều regularization hơn (mô hình đơn giản hơn).

39. Bias-variance decomposition là gì?

Bias-variance decomposition phân tách expected generalization error của một mô hình thành ba phần: Bias, Variance, và Irreducible Error.

$$E[(y - \hat{f}(x))^2] = \text{Bias}^2[\hat{f}(x)] + \text{Variance}[\hat{f}(x)] + \sigma^2$$

Interpretation:

- **High Bias:** Mô hình quá đơn giản $\rightarrow$ underfitting.
- **High Variance:** Mô hình quá phức tạp $\rightarrow$ overfitting.
- **Best model:** Đạt được sự cân bằng hợp lý $\rightarrow$ tối thiểu hóa tổng lỗi.

Exam Tip

Bias liên quan đến các giả định của mô hình.
Variance liên quan đến độ nhạy cảm của mô hình.
Irreducible error là nhiễu mà ta không thể loại bỏ.

40. Support vector machine (SVM) là gì?

SVM là một thuật toán supervised learning được dùng cho classification và regression. Nó hoạt động bằng cách tìm hyperplane tối ưu phân tách các lớp với margin lớn nhất.

Key Ideas:

- Tìm hyperplane với margin lớn nhất.
- Chỉ sử dụng các support vector (điểm gần nhất) đến hyperplane.
- Có thể xử lý dữ liệu phi tuyến bằng kernel trick.

Types:

- Linear SVM: Cho dữ liệu phân tách tuyến tính.
- Kernel SVM: Cho dữ liệu phi tuyến (vd, RBF kernel, polynomial kernel).

Margin: Khoảng cách giữa hyperplane và điểm dữ liệu gần nhất của bất kỳ lớp nào.

Exam Tip

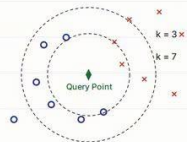
SVM tập trung vào ranh giới (boundary), không phải các điểm riêng lẻ.
Margin càng lớn $\rightarrow$ generalization càng tốt.

41. K-nearest neighbors (KNN) là gì?

KNN là một thuật toán instance-based learning đơn giản, được dùng cho classification và regression. Nó đưa ra dự đoán dựa trên "k" ví dụ huấn luyện gần nhất.

How it works:

1. Chọn giá trị k.
2. Tính khoảng cách từ query point đến tất cả các training point.
3. Chọn k neighbor gần nhất.
4. Với classification: dùng majority vote của các neighbor. Với regression: dùng trung bình (hoặc trung bình có trọng số) của các neighbor.



○ Class A × Class B ◆ Query Point

Exam Tip

KNN là lazy learning (không có giai đoạn huấn luyện). Tại thời điểm dự đoán, nó nhìn vào training data.

42. Naive Bayes là gì?

Naive Bayes là một thuật toán classification xác suất, dựa trên Bayes' theorem với giả định rằng các feature độc lập có điều kiện (conditionally independent).

$$P(C|X) = \frac{P(X|C) P(C)}{P(X)}$$

where: C = class, X = features, $P(C|X)$ = posterior, $P(X|C)$ = likelihood, $P(C)$ = prior, $P(X)$ = evidence

How it works:

- Tính prior $P(C)$ cho mỗi class.
- Tính likelihood $P(X|C)$ với giả định các feature độc lập.
- Dùng Bayes' theorem để tính posterior.
- Dự đoán class có posterior probability cao nhất.

Types: Gaussian Naive Bayes (dữ liệu liên tục), Multinomial Naive Bayes (dữ liệu đếm rời rạc), Bernoulli Naive Bayes (dữ liệu nhị phân).

Exam Tip

"Naive" nghĩa là nó giả định các feature độc lập, điều này thường không đúng trong thực tế → nhưng nó vẫn hoạt động tốt!

43. Principal component analysis (PCA) là gì?

PCA là một kỹ thuật dimensionality reduction, chuyển đổi dữ liệu vào một hệ tọa độ mới, trong đó variance lớn nhất nằm trên principal component đầu tiên, lớn thứ hai nằm trên component thứ hai, v.v.

Steps:

1. Chuẩn hóa dữ liệu (Standardize).
2. Tính covariance matrix.
3. Tính eigenvalues và eigenvectors.
4. Sắp xếp eigenvector theo eigenvalue giảm dần.
5. Chọn top k eigenvector.
6. Chiếu (project) dữ liệu lên subspace mới.

Exam Tip

PCA là unsupervised. Nó bảo toàn variance, không phải nhãn lớp. Rất tốt cho visualization!

44. Overfitting là gì và làm sao để ngăn chặn nó?

Overfitting xảy ra khi một mô hình học dữ liệu huấn luyện quá kỹ, bao gồm cả nhiễu, và hoạt động kém trên dữ liệu chưa từng thấy.

Signs of Overfitting:

- Training error rất thấp.
- Validation/test error cao.

How to prevent?

- Sử dụng nhiều training data hơn.
- Đơn giản hóa mô hình.
- Áp dụng regularization (L1/L2).
- Sử dụng dropout (cho neural network).
- Early stopping.
- Cross-validation.



45. Confusion matrix là gì và vì sao nó quan trọng?

Confusion matrix là một bảng được dùng để đánh giá hiệu năng của một mô hình classification bằng cách so sánh nhãn dự đoán với nhãn thực tế.

	Predicted Positive	Predicted Negative
Actual Positive	TP (True Positive)	FN (False Negative)
Actual Negative	FP (False Positive)	TN (True Negative)

Why important?

- Cho bức tranh chi tiết về hiệu năng mô hình.
- Hữu ích khi các lớp bị mất cân bằng.
- Cho thấy các loại lỗi mà mô hình mắc phải.
- Là cơ sở cho các metric như Accuracy, Precision, Recall, F1 Score, v.v.

46. Sự khác biệt giữa bagging và boosting là gì?

Cả hai đều là kỹ thuật ensemble nhưng khác nhau về cách tiếp cận.

Aspect	Bagging	Boosting
Full Form	Bootstrap Aggregating	Boosting
Approach	Song song (các mô hình độc lập)	Tuần tự (các mô hình phụ thuộc mô hình trước)
Training Data	Tập con ngẫu nhiên (có lặp lại) của dữ liệu	Cùng dữ liệu, tập trung vào mẫu bị phân loại sai
Combines	Trung bình (regression) hoặc Majority vote (classification)	Kết hợp có trọng số của các mô hình
Examples	Random Forest, Bagged Trees	AdaBoost, Gradient Boosting, XGBoost
Effect	Giảm variance	Giảm bias (và variance)

47. Gradient descent là gì?

Gradient Descent là một thuật toán tối ưu hóa được dùng để tối thiểu hóa loss function bằng cách lặp lại việc di chuyển theo hướng ngược với gradient.

How it works:

1. Bắt đầu với trọng số khởi tạo (ngẫu nhiên).
2. Tính gradient của loss.
3. Cập nhật trọng số theo hướng ngược lại của gradient.
4. Lặp lại cho đến khi hội tụ.

Update Rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \nabla L(w)$$

trong đó: w = trọng số, η = learning rate, $\nabla L(w)$ = gradient của loss

Exam Tip

Learning rate (η) rất quan trọng. Quá cao $\rightarrow$ overshoot. Quá thấp $\rightarrow$ học chậm. Vừa đủ $\rightarrow$ hội tụ nhanh.

48. Dimensionality reduction là gì và vì sao nó được sử dụng?

Dimensionality reduction là quá trình giảm số lượng input feature trong một tập dữ liệu trong khi vẫn giữ lại càng nhiều thông tin quan trọng càng tốt.

Why is it used?

- Giảm chi phí tính toán.
- Giúp loại bỏ nhiễu và sự dư thừa.
- Giảm thiểu curse of dimensionality.
- Cải thiện hiệu năng mô hình (giảm overfitting).
- Hỗ trợ visualization.

Technique	Type	Key Idea
PCA	Linear	Chiếu dữ liệu lên các principal component (variance tối đa).
SVD	Linear	Phân rã ma trận thành U, Σ, V^T và giữ lại các component hàng đầu.
t-SNE	Non-linear	Bảo toàn cấu trúc cục bộ ở chiều thấp hơn.
UMAP	Non-linear	Bảo toàn cấu trúc cục bộ và toàn cục hiệu quả.

Exam Tip

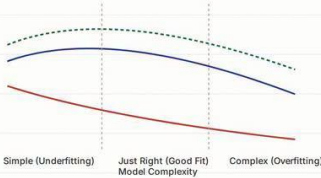
Dùng PCA cho tốc độ và khả năng diễn giải. Dùng t-SNE/UMAP để visualization dữ liệu phức tạp, phi tuyến.

49. Bias-variance tradeoff với biểu đồ minh họa là gì?

Bias-variance tradeoff là sự cân bằng giữa bias (lỗi từ các giả định sai) và variance (lỗi từ sự nhạy cảm với training data).

Bias (High bias):

- Mô hình quá đơn giản.
- Bị underfitting dữ liệu.
- Training error cao và test error cao.



Variance (High variance):

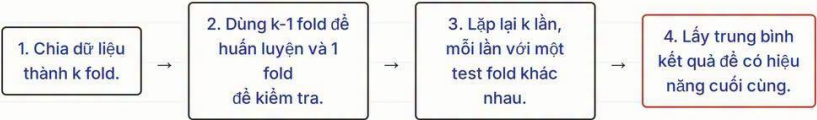
- Mô hình quá phức tạp.
- Bị overfitting dữ liệu.
- Training error thấp nhưng test error cao.

Goal: Tìm sự cân bằng hợp lý để tối thiểu hóa tổng lỗi.

50. Cross-validation là gì và có những loại nào?

Cross-validation là một kỹ thuật để đánh giá một mô hình bằng cách chia dữ liệu thành nhiều fold và huấn luyện/kiểm tra mô hình nhiều lần.

How it works? (k-Fold Cross-Validation)



Types of Cross-Validation:

- Hold-Out CV: Chia một lần duy nhất (vd, 80% train, 20% test). Đơn giản nhưng variance cao.
- K-Fold CV: Dữ liệu được chia thành k fold. Ổn định hơn và được sử dụng rộng rãi.
- Stratified K-Fold: Bảo toàn phân phối lớp trong mỗi fold. Tốt cho dữ liệu mất cân bằng.
- Leave-One-Out CV (LOOCV): $k = n$ (số lượng samples). Bias thấp, variance cao, tốn kém.
- Time Series CV: Tôn trọng thứ tự thời gian. Huấn luyện trên quá khứ, kiểm tra trên tương lai.